

Práctico Nº 3: Programación OO

Ejercicio 1: Procesamiento con Stream

Considere la definición de la clase **Producto** con: *nombre:String*, *precio:float* y *stock:int*; con *setter()* y *getter()*. Se pide analizar el siguiente código; escribir el resultado que se calcula en cada caso.

```
public static void main(String[] args){
    List<Producto> lista = Arrays.asList(
        new Producto("Azucar Negra", 500, 20),
        new Producto("Azucar", 400, 80),
        new Producto("Cafe Tostado", 1500, 50),
        new Producto("Yerba Mate", 1800, 150),
        new Producto("Cafe Torrado", 2100, 40));

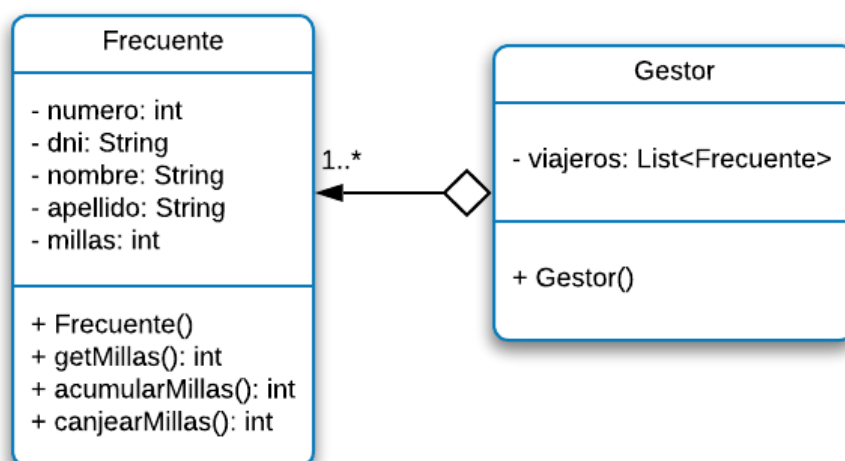
    lista.stream().filter(x -> x.getStock() > 10).findFirst().ifPresent(x ->
        System.out.println(x.getNombre()));

    System.out.println( lista.stream().count() );
    System.out.println(lista.stream().filter(x -> x.getStock() == 0).count());
    System.out.println(lista.stream().filter(x -> x.getNombre().contains("Cafe")) .count());
    lista.stream().map(x -> x.getPrecio()).reduce(Float::min).ifPresent(System.out::println);
}
```

Ejercicio 2: Viajeros

Descripción General: En una aerolínea ofrece una promoción a sus viajeros frecuentes que consiste en acumular millas por los viajes que realizan, pudiendo luego recibir beneficios a través del canje de millas.

Usted trabaja en el área de sistemas de la aerolínea y le han solicitado la implementación de un sistema capaz de gestionar la promoción. Respetando el siguiente diseño de clase:



Descripción de los métodos de la clase Frecuente:

- El constructor.
- Getter y Setter de los atributos
- "acumularMillas", recibe como parámetro la cantidad de millas recorridas, las suma en el atributo correspondiente y retorna el valor del atributo actualizado.
- "canjearMillas", recibe como parámetro la cantidad de millas a canjear. Para canjear las millas debe verificarse que la cantidad a canjear sea menor o igual a la cantidad de

Práctico Nº 3: Programación OO

millas acumuladas, caso contrario mostrar un mensaje de error en la operación. Retorna el valor de la cantidad de millas acumuladas.

Implemente:

1. Las clases dadas en el diagrama.
2. Un programa que presente un menú con las siguientes funcionalidades:
 - a) Cargar viajero: registrar un nuevo viajero.
 - b) Mostrar viajero: dado el número de un viajero mostrar todos sus datos (use el método toString).
 - c) Consultar Cantidad de Millas: se ingresa un dni y si corresponde a un viajero registrado, retorna la cantidad de millas del viajero.
 - d) Acumular Millas: se ingresa un dni y cantidad de millas, si corresponde a un viajero registrado se incrementa las millas acumuladas del viajero en la cantidad de millas dadas.
 - e) Canjear Millas: se ingresa un dni y cantidad de millas, si corresponde a un viajero registrado se decrementa las millas acumuladas del viajero en la cantidad de millas dadas.
 - f) Mejor viajero: Mostrar los datos del/ los viajeros con mayor cantidad de millas.

Ejercicio 3: Manejo de errores

Implementar validaciones utilizando excepciones, de modo que el programa pueda detectar situaciones inválidas, informar al usuario mediante mensajes claros y continuar su ejecución sin detenerse. Considerar las siguientes situaciones:

- a) "Cargar Viajero": Si se intenta cargar un viajero con dni existente → lanzar una excepción.
- b) "Opciones del menú que piden el DNI": Si el DNI ingresado no corresponde a un viajero registrado → lanzar una excepción.

Ejercicio 4: Procesamiento concurrente

Objetivo: Desarrollar un programa que procese texto utilizando **programación concurrente**, con el fin de mejorar el rendimiento en el análisis de datos.

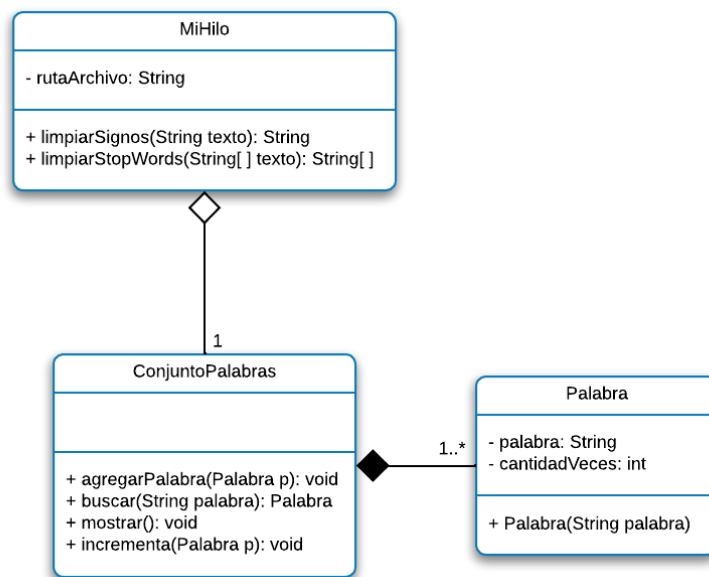
Descripción: En ciencia de datos, una tarea frecuente en el procesamiento de texto consiste en contar cuántas veces aparece cada palabra dentro de un texto. Para realizar este análisis correctamente, primero es necesario limpiar el contenido.

La limpieza del texto incluye:

- Eliminar caracteres que no aportan información relevante, como signos de puntuación, comillas y otros símbolos.
- Quitar las *stopwords*, es decir, palabras de uso muy frecuente que no aportan valor al análisis, por ejemplo: artículos, preposiciones y conjunciones.

Diagrama de clases (incompleto)

Práctico Nº 3: Programación OO



Consigna:

Construir un programa que procese en paralelo dos archivos de texto (.txt) y determine cuántas veces aparece cada palabra teniendo en cuenta la información contenida en ambos archivos. Para ello, se pide:

1. Crear dos instancias de la clase MiHilo.
2. Cada hilo debe:
 - a) Leer un archivo de texto.
 - b) Limpiar el texto (eliminar caracteres innecesarios y stopwords).
 - c) Registrar la frecuencia de cada palabra.
 - d) Almacenar los resultados en una instancia de la clase ConjuntoPalabras.
3. Una vez finalizados ambos hilos:
 - a) Combinar los resultados obtenidos por cada hilo.
 - b) Mostrar por pantalla cada palabra junto con la cantidad total de apariciones en ambos archivos.

Ejemplo:

Texto1.txt

*Como cuesta, subir la cuesta.
Si la cuesta no fuera cuesta, no
costaría.*

Texto2.txt

La cuesta no costaría si no fuera cuesta.
Pero como es cuesta, cuesta mucho.

Salida del programa:

Palabra: como cantidad: 2
 Palabra: cuesta cantidad: 8
 Palabra: subir cantidad: 1
 Palabra: no cantidad: 4
 Palabra: si cantidad: 2
 Palabra: fuera cantidad: 2
 Palabra: pero cantidad: 1
 Palabra: es cantidad: 1
 Palabra: mucho cantidad: 1

Práctico Nº 3: Programación OO

Información de interés para resolver el ejercicio:

- Diapositivas de la cátedra.
- Métodos `toLowerCase`, `split`, `replace` de la clase `String`.
- <https://docs.oracle.com/javase/tutorial/essential/io/charstreams.html>
- <https://docs.oracle.com/javase/tutorial/essential/io/buffers.html>

Ejercicio 5:

Implementar una solución alternativa donde ambos hilos compartan una misma instancia de `ConjuntoPalabras`, utilizando mecanismos de sincronización para evitar inconsistencias.

Propuesto

Considere la definición de la clase `Producto` con: `nombre:String`, `categoría:String` y `precio:float`; con `setter()`, `getter()` y `toString()`. Se pide analizar e interpretar el siguiente código; escribe el resultado que aparece en pantalla.

```
public static void main(String[] args){
```

```
    List<Producto> productos = Arrays.asList(  
        new Producto("Laptop", "Electrónica", 1200.0),  
        new Producto("Smartphone", "Electrónica", 800.0),  
        new Producto("Camiseta", "Ropa", 30.0),  
        new Producto("Pantalón", "Ropa", 50.0),  
        new Producto("Libro", "Libros", 20.0) );
```

1)

```
    System.out.println(productos.stream().filter(x -> x.getNombre().startsWith("L"))  
                                .map(x -> x.getNombre().toUpperCase())  
                                .collect(toList()));
```

2) Nota: Aplicar la misma semántica que en una BD

```
    Map<String, Double> listado = productos.stream()  
        .collect(Collectors.groupingBy(Producto::getCategoria,  
                                       Collectors.averagingDouble(Producto::getPrecio)));  
  
    System.out.println(listado);
```